

# DataMaster - Pipeline de Dados para Engajamento em Mídias Sociais

---

## I. Objetivo do Case

Repositório: <https://github.com/diegomeyer/datamaster.git>

O objetivo deste case é construir um pipeline de dados robusto, escalável e manutenível para processar e analisar dados de engajamento de múltiplas plataformas de mídia social (Instagram, Facebook, X/Twitter). O pipeline transforma dados brutos em insights acionáveis, permitindo responder perguntas como: - Qual o engajamento diário (posts, likes, compartilhamentos, comentários) em cada plataforma? - Quem são os influenciadores com maior engajamento? - Quais os horários de pico para postagens?

A arquitetura segue o padrão **Data Lakehouse** (Medallion Architecture: Bronze, Silver, Gold), garantindo governança, qualidade e performance.

---

## II. Arquitetura de Solução e Arquitetura Técnica

### Visão Geral

O pipeline é composto por: 1. **Geração de Dados Sintéticos**: Simula posts de redes sociais usando Python e Faker. 2. **Ingestão em Tempo Real**: Dados enviados para tópicos Kafka. 3. **Processamento Distribuído**: PySpark consome do Kafka e grava dados brutos (Bronze) no MinIO (S3 compatível). 4. **Transformação e Limpeza**: Dados Bronze são processados e normalizados (Silver). 5. **Agregação e KPIs**: Dados Silver são agregados para geração de métricas de negócio (Gold). 6. **Orquestração**: Airflow agenda e automatiza os fluxos. 7. **Monitoramento**: Prometheus coleta métricas e Grafana exibe dashboards.

### Tecnologias Utilizadas

A arquitetura é organizada por camadas funcionais. O diagrama acima apresenta os principais serviços envolvidos, agrupados por suas responsabilidades.

| Tecnologia            | Função na Arquitetura        |
|-----------------------|------------------------------|
| <b>Docker Compose</b> | Orquestração dos containers  |
| <b>Kafka</b>          | Mensageria em tempo real     |
| <b>Spark</b>          | Processamento distribuído    |
| <b>S3 (MinIO)</b>     | Data Lake (armazenamento S3) |
| <b>Airflow</b>        | Orquestração de pipelines    |

| Tecnologia            | Função na Arquitetura                     |
|-----------------------|-------------------------------------------|
| <b>Prometheus</b>     | Coleta de métricas                        |
| <b>Grafana</b>        | Dashboards de métricas                    |
| <b>Jupyter</b>        | Exploração/análise de dados               |
| <b>PostgreSQL</b>     | Banco de dados do Airflow                 |
| <b>Kafka Exporter</b> | Exporta métricas do Kafka para Prometheus |

### Por que cada tecnologia?

#### Detalhes sobre o uso do MinIO

- **MinIO** é utilizado como armazenamento S3 compatível, simulando um ambiente cloud localmente.
- Todos os dados processados (Bronze, Silver, Gold) são salvos em buckets S3, permitindo fácil integração futura com AWS S3 ou outros provedores.
- O acesso ao S3 é feito via credenciais (AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY) e endpoint configurados nos serviços Spark, Airflow e Jupyter.
- O particionamento dos dados no S3 facilita consultas analíticas e otimiza custos de armazenamento.
- O uso do S3 permite escalabilidade horizontal e separação clara das camadas de dados.

#### Detalhes sobre o uso do Apache Iceberg

- **Transações ACID:** Garante consistência e integridade dos dados mesmo em operações concorrentes, evitando corrupção de tabelas.
- **Time Travel:** Permite consultar versões anteriores dos dados, facilitando auditoria, debugging e recuperação de informações.
- **Schema Evolution:** Suporta alterações de esquema (adicionar/remover/renomear colunas) sem downtime ou migração complexa.
- **Partition Evolution:** Permite alterar a estratégia de particionamento sem recriar tabelas, otimizando consultas ao longo do tempo.
- **Performance:** Metadados otimizados e pruning inteligente aceleram queries, reduzindo leitura desnecessária de arquivos.
- **Compatibilidade:** Integra-se nativamente com Spark, Flink, Trino, Presto, Hive e ferramentas modernas de analytics.
- **Gerenciamento de Dados:** Suporta operações de expurgo, compactação e otimização de arquivos, reduzindo custos e melhorando performance.

#### Vantagens no contexto do projeto

- **Governança e rastreabilidade:** O Iceberg facilita o controle de versões e auditoria dos dados, essencial para compliance e LGPD.

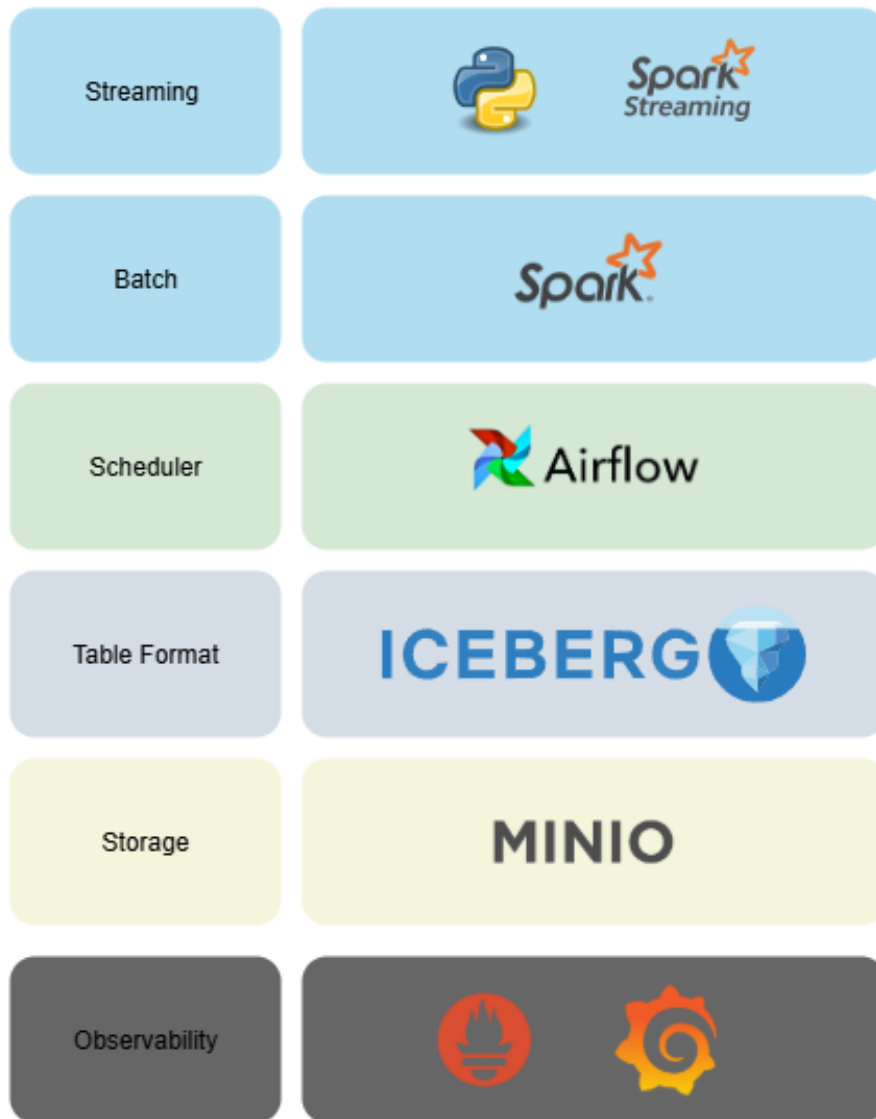


Figure 1: docs/datamaster-resumido.png

- **Escalabilidade:** Projetado para petabytes de dados, permite crescimento do Data Lake sem perda de performance.
  - **Preparação para produção:** Usar Iceberg localmente (com MinIO) facilita a migração para ambientes cloud (AWS S3, GCP, Azure) sem mudanças de código.
- 

**Detalhes sobre o uso do Kafka** O **Kafka** é utilizado como sistema de mensageria para ingestão de dados em tempo real. Ele atua como intermediário entre os produtores (geração de dados sintéticos) e os consumidores (jobs Spark de ingestão e processamento).

**Por que usar Kafka?** - **Desacoplamento:** Permite que produtores e consumidores operem de forma independente, aumentando a flexibilidade do pipeline. - **Escalabilidade:** Kafka suporta alto volume de mensagens e múltiplos consumidores em paralelo. - **Persistência e tolerância a falhas:** Mensagens ficam armazenadas nos tópicos até serem processadas, garantindo resiliência. - **Monitoramento:** Combinado ao Kafka Exporter, permite acompanhar métricas como lag, throughput e saúde dos tópicos.

**Vantagens no contexto do projeto** - Permite ingestão contínua e paralela dos dados de diferentes redes sociais. - Facilita a escalabilidade horizontal do pipeline. - Garante que nenhum dado seja perdido mesmo em caso de falhas temporárias nos consumidores.

---

**Detalhes sobre o uso do Apache Airflow** O **Apache Airflow** é responsável por orquestrar e automatizar todo o pipeline de dados, controlando dependências, agendamento e monitoramento das tarefas.

**Por que usar Airflow?** - **Orquestração visual:** Permite desenhar, monitorar e reexecutar pipelines complexos via interface web. - **Agendamento flexível:** Suporta execuções baseadas em tempo, eventos ou dependências entre tarefas. - **Extensibilidade:** Suporta operadores customizados, integração com Spark, S3, bancos de dados e muito mais. - **Persistência:** Utiliza PostgreSQL para armazenar logs, histórico e metadados das execuções.

**Vantagens no contexto do projeto** - Garante rastreabilidade e reprodutibilidade dos fluxos de dados. - Permite a rápida identificação e reexecução de tarefas com falha. - Facilita a integração e automação de todo o ciclo de ingestão, processamento e agregação.

---

**Detalhes sobre o uso do Prometheus** O **Prometheus** é utilizado para coleta e armazenamento de métricas de todos os serviços do pipeline (Kafka, Spark, Airflow, etc).

**Por que usar Prometheus?** - **Coleta automática:** Scrape de métricas expostas por exporters e serviços. - **Consulta poderosa:** Linguagem de consulta própria (PromQL) para análises detalhadas. - **Alertas:** Permite configurar alertas automáticos para eventos críticos.

**Vantagens no contexto do projeto** - Proporciona visibilidade em tempo real da saúde e performance dos componentes. - Permite identificar gargalos, falhas e otimizar recursos do pipeline. - Integração direta com Grafana para visualização.

---

**Detalhes sobre o uso do Grafana** O Grafana é utilizado para visualização e criação de dashboards interativos com as métricas coletadas pelo Prometheus.

**Por que usar Grafana?** - **Dashboards customizáveis:** Permite criar painéis visuais para monitorar KPIs, uso de recursos, lags, etc. - **Alertas visuais:** Notificações em tempo real para eventos críticos. - **Integração:** Suporte nativo ao Prometheus e outros bancos de dados de métricas.

**Vantagens no contexto do projeto** - Facilita o acompanhamento visual do funcionamento do pipeline. - Permite rápida identificação de problemas e tomada de decisão baseada em dados. - Dashboards podem ser compartilhados com toda a equipe para colaboração e transparência.

---

### III. Explicação sobre o Case Desenvolvido

Arquitetura detalhada

#### 1 - Geração de Dados Fakes

- O serviço `api_fake/main.py` gera continuamente dados sintéticos que simulam posts das redes sociais Facebook, Instagram e X.
- Ele utiliza geradores de dados específicos para cada plataforma (ex: `facebook_generator.py`) e os publica em seus respectivos tópicos no Kafka: `instagram-post`, `facebook-post`, `x-post`.

#### 2 - Ingestão para a Camada Bronze

- Três jobs do Spark Streaming `run_ingestion.py` são iniciados, um para cada rede social.
- Cada job consome as mensagens de seu tópico Kafka específico, adiciona metadados de ingestão (como `ingestion_date`) e escreve os dados brutos como tabelas Iceberg na camada Bronze do Data Lake, particionados por data.

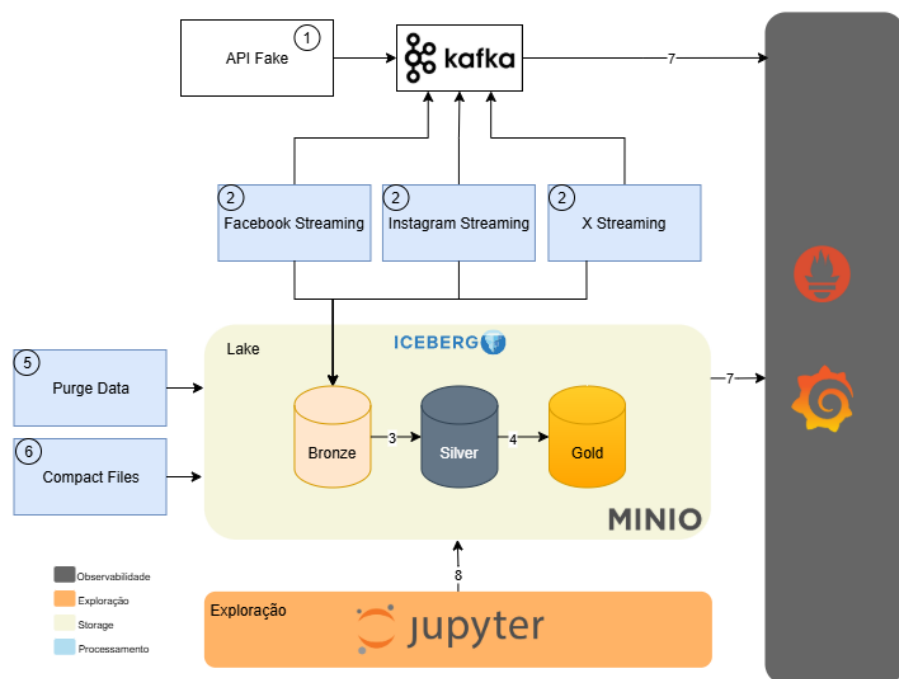


Figure 2: docs/datamaster\_detalhado.png

- O processo utiliza checkpoints para garantir a tolerância a falhas e o processamento “exactly-once”.

### 3 - Transformação para a Camada Silver

- Uma DAG do Airflow `silver_unified_dag.py` orquestra a execução de 10 em 10 minutos do script `silver_unified_batch.py`.
- Este script lê os dados brutos de todas as fontes da camada Bronze para uma data de processamento específica.
- Ele aplica transformações para limpar, normalizar e unificar os dados em um schema comum, armazenando o resultado na tabela Iceberg `hadoop.silver.social_media`.
- Nesta fase, o campo `author` é anonimizado usando uma função de hashing (SHA-256 com salt) para proteger a identidade dos usuários, em conformidade com as boas práticas da LGPD.

### 4 - Agregação para a Camada Gold

- Uma DAG do Airflow `silver_to_gold_dag.py` orquestra a execução diária do script `silver_to_gold_batch.py`.
- Este job consome os dados unificados da camada Silver e realiza agregações para gerar KPIs de negócio, como:
  - Engajamento diário (total de posts, likes, compartilhamentos, comentários) por plataforma, na tabela Iceberg `hadoop.gold.daily_engagement`.
  - Ranking de autores com maior engajamento, na tabela Iceberg `hadoop.gold.top_authors`.
  - Distribuição de postagens por hora do dia, na tabela Iceberg `hadoop.gold.hourly_posts`.
- Os resultados são salvos em tabelas específicas na camada Gold, prontos para serem consumidos por ferramentas de análise.

### 5 - Expurgo de Dados

- Para governança de dados, uma DAG mensal `purge_bronze_dag.py` é responsável por executar o script `purge_bronze_data.py`, que expurga dados antigos da camada Bronze, aplicando uma política de retenção de 90 dias.

### 6. Compactação de Small Files nas Tabelas Iceberg

- Para garantir a performance e evitar o acúmulo de arquivos pequenos (“small files”) nas tabelas Iceberg do Data Lake, o projeto conta com uma DAG dedicada de compactação automática.

- A DAG `compact_iceberg_small_files_dag.py` utiliza o `PythonOperator` para executar periodicamente o script `compact_iceberg_small_files_batch.py`, que realiza a operação de `rewrite_data_files` em todas as tabelas das camadas Bronze, Silver e Gold.
- Essa compactação reduz a quantidade de arquivos pequenos no S3/MinIO, melhorando a performance das consultas e otimizando custos de armazenamento.
- O processo é totalmente automatizado e pode ser facilmente ajustado para incluir novas tabelas ou alterar a periodicidade conforme a necessidade do projeto.

## 7 - Envio de Metrica e Dashboards

- Kafka Exporter envia metricas relacionadas ao Kafka para o Prometheus
- Minio enviam metricas para o prometheus.
- As metricas são consumidas via Dashboard no Grafan

## 8 - Ferramenta de Exploração de Dados

- A adoção do Jupyter é motivada pela necessidade de um ambiente interativo para exploração de dados e desenvolvimento ágil de análises. Ele permite que cientistas e engenheiros consultem o data lakehouse de forma iterativa, validem hipóteses e prototipem lógicas complexas com feedback visual e imediato antes da produção.

## Estrutura do Data Lake

- **Bronze:** Dados brutos, exatamente como recebidos.
- **Silver:** Dados limpos, normalizados e enriquecidos.
- **Gold:** Dados agregados e métricas de negócio, otimizados para consumo.

## Orquestração e Monitoramento

- O Airflow agenda e monitora os fluxos de ingestão, transformação e agregação.
- O Prometheus coleta métricas de performance dos containers e jobs.
- O Grafana exibe dashboards de monitoramento.
- O Kafka Exporter expõe métricas detalhadas do Kafka para Prometheus.

**Detalhamento do Monitoramento** Kafka, MinIO expõe métricas para o Prometheus, que são visualizadas no Grafana

Métricas para Kafka

Métricas MiniO

Com essas métricas é possível acompanhar a saúde das aplicações e o fluxo de mensagens. ### Testes





Figure 3: docs/kafka-dash.png

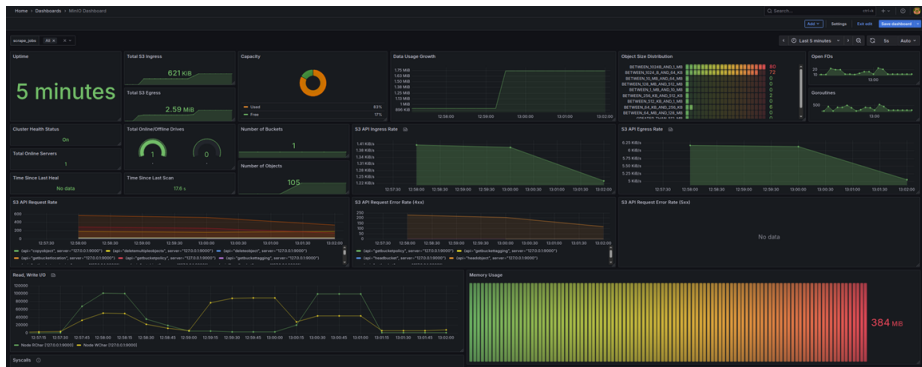


Figure 4: docs/minio-dash.png

O projeto conta com testes unitários para garantir a qualidade dos componentes. Para executá-los, defina primeiro a variável de ambiente `PYTHONPATH` para o diretório do componente a ser testado e depois use o `pytest`.

- Testes unitários para API Fake em `api_fake/tests`.
  - Para rodar os testes: `bash export PYTHONPATH=$(pwd)/api_fake pytest --cov=api_fake api_fake/tests/ -v`
  - Testes unitários para o processo de Streaming em `streaming_ingestion/tests`.
  - Para rodar os testes: `bash export KAFKA_BOOTSTRAP_SERVERS=teste export PYTHONPATH=$(pwd)/streaming_ingestion pytest --cov=streaming_ingestion streaming_ingestion/tests/ -v`
  - Testes unitários para as dags do airflow em `airflow/tests`.
  - Para rodar os testes: `bash export PYTHONPATH=$(pwd)/airflow/dags pytest --cov=airflow airflow/tests/ -v`
- 

## IV. Execução, Melhorias e Considerações Finais

### Como Executar o Projeto

#### 1. Clone o repositório

```
git clone https://github.com/diegomeyer/datamaster.git
cd datamaster
```

#### 2. Suba toda a infraestrutura

```
./start_services.sh
```

Isso irá iniciar todos os containers necessários (Kafka, Spark, MinIO, Airflow, Prometheus, Grafana, etc).

#### 3. Acompanhe o pipeline

- Use o Airflow (porta 8082) para monitorar as DAGs de ingestão e transformação.
- Visualize métricas e dashboards no Grafana (porta 3000, usuário/senha: admin/admin).
- O Jupyter estará disponível na porta 8888 para exploração dos dados.

### Melhorias Sugeridas

- **Escalabilidade:** Particionamento dos tópicos Kafka, paralelismo no PySpark, expansão do cluster MinIO.
- **Observabilidade:** Integração com OpenTelemetry para rastreamento distribuído.
- **Governança:**

- Implementar políticas de retenção e expurgo automático de dados em todas as camadas (Silver e Gold), não apenas na Bronze.
- Adotar um catálogo de dados para documentar e facilitar a descoberta dos ativos de dados.
- **Segurança:** Autenticação/autorização no Kafka, criptografia de dados em repouso, uso de cofre de segredos (ex: HashiCorp Vault).
- **LGPD:** Em caso de dados reais, aplicar anonimização (generalização, supressão, k-anonimidade, tokenização).
- **Qualidade de Dados:** Implementar uma ferramenta como Great Expectations para adicionar validações automáticas e testes de qualidade nos dados em cada camada do pipeline, garantindo a integridade e a confiabilidade das informações.

### Considerações Finais

A solução entrega um pipeline moderno, escalável e observável para ingestão, processamento e análise de dados em tempo real, utilizando tecnologias consolidadas no mercado. As melhorias sugeridas podem elevar ainda mais a robustez e governança da plataforma, tornando-a apta a desafios de Big Data e requisitos de compliance.

---